# Semivra Libellus - Application Status Report

_Generated 2026-07-04  Branch `fixes/prod-hardening`  Verified against a live run + full test suites_

---

## 1. Project overview

**Semivra Libellus** is a full-stack business operations system that combines a Point-of-Sale, inventory management, and real double-entry accounting in one web application. It runs in two configurable modes from the same codebase (`BUSINESS_TYPE`):

- **`fb` - Food & Beverage:** caf/restaurant POS with QR table ordering, kitchen flow, and recipe-based ingredient costing.
- **`log` - Logistics / distribution (this deployment, "Kasa Lokal"):** order dispatch, storage-room stock view, and a **client portal** where business customers log in and place orders directly.

It is designed for the Philippine market (Non-VAT percentage tax, BIR-style references, peso denominations), works on tablets and phones, keeps taking orders when the internet drops (offline PWA queue), and is architected for **white-label resale** - with multi-tenancy work underway so several businesses can eventually share one deployment.

**Stack:** React 18 + Vite + Tailwind (frontend, Vercel)  Node/Express + MongoDB Atlas + Socket.io (backend, Railway)  Docker/nginx configs for self-hosting.

---

## 2. What the app does - in plain English (for non-technical readers)

**Selling and orders**

- A cashier register: search products, apply discounts, take cash/e-wallet/bank payments, compute change from peso denominations, print receipts.
- Orders move through clear stages (Preparing -> Ready -> Delivered/Picked-Up), including partial deliveries and a dispatch tracker for deliveries.
- Customers can order by themselves: scan a QR code (F&B mode) or log into a **client portal** (logistics mode). Every screen updates live on all devices at once.

**Stock**

- The app knows your recipes/products, so every sale automatically subtracts the right ingredients or items from stock.
- It tracks expiry dates batch-by-batch (oldest first), warns you when stock is low or expiring, records spoilage with a required reason, and supports Excel import for bulk stock counts.

**Money - the books keep themselves**

- Every sale, void, expense, and spoilage automatically writes a proper accounting entry (double-entry, always balanced). You get a real Profit & Loss, Balance Sheet, Accounts Receivable/Payable, and a general ledger - exportable to CSV/PDF/Excel - without a bookkeeper re-typing anything.
- Philippine Non-VAT (3% percentage tax) rules are built in.

**People and cash control**

- Staff log in with their own accounts, declare starting cash, clock in, and reconcile the drawer at end of shift (expected vs. actual, variance tracked per cashier).
- The owner (superadmin) has extra powers: voids, settings, audit reports, user management - and is excluded from staff statistics.

**Reports**

- Live dashboard: revenue, best sellers, inventory value, out-of-stock count, daily revenue trend, plus exportable analytics and audit reports.

---

## 3. What the app does - technical summary (for developers)

- **Auth:** dual-token - 15-minute access JWT held in memory only + opaque refresh token in an httpOnly/Secure/SameSite cookie, sha256-hashed server-side in a TTL `RefreshSession` collection with real revocation (logout, password/role change, deletion). Origin-allowlist guard on auth routes. bcrypt cost 12.
- **Validation & hardening:** Helmet, Zod request validation (strips unknown keys, 422 field errors), regex-injection/ReDoS escaping, rate limits (login 5/15min, orders 60/min, API 300/min), production error-detail suppression, structured pino logging, optional Sentry.
- **Data integrity:** atomic inventory deduction (`findOneAndUpdate $inc`), atomic order-number sequencing, balanced-journal assertion (throws if DRCR), money/inventory routes wrapped in MongoDB transactions, completed-order immutability, idempotency-key double-submit protection.
- **Multi-tenancy (in progress):** Phase 1 foundation + Phase 2a (tenant identity in auth) + Phase 2b partial (fallback-safe read scoping by `businessType`, with a one-time backfill migration on boot) - all shipped non-breaking.
- **Real-time:** Socket.io events (`erpUpdated`, order events) keep every connected device in sync.
- **Client:** route-level lazy loading, code-split heavy libs (jsPDF/xlsx load on demand), error boundary, PWA offline order queue, wake-lock, responsive sidebar-drawer layout with horizontally scrollable data tables.
- **Ops:** `/health` endpoint, graceful shutdown, fail-fast boot (refuses to start in production with weak JWT_SECRET / default ADMIN_PASS / missing ALLOWED_ORIGINS), backup/restore scripts, Makefile, Docker, GitHub Actions CI, `GO_LIVE.md` runbook.

---

## 4. Actual state - verified 2026-07-03/04 (not just claimed)

| Check | Result |
|---|---|
| Server test suite (22 files: auth, money, ledger, tenancy, sockets, fault injection, edge cases) | 339/339 passed |
| Playwright browser E2E (real Chrome: auth flows incl. reload-spam, every dashboard tab, ledger sub-views, zero-console-error assertions) | 9/9 passed |
| ESLint (client + server) | clean |
| Production build (`vite build`) | clean |
| `npm audit` | 0 known production vulnerabilities (per last audit) |
| Live manual run (login -> Orders, Inventory, Ledger, Analytics with real data) | no errors |
| Responsiveness (375px phone / 768px tablet / desktop) | all layouts verified working |

E2E now runs safely with one command against a throwaway in-memory database: `npm run e2e:server` + `npm run e2e` (added 2026-07-03).

---

## 5. Is it ready for deployment?

**Verdict: the code is production-ready; the *environment and process* around it still has open items.** Nothing in the application itself blocks a go-live.

**Must do before/at deploy (blockers):**

1. Set production env vars on the server host: strong `JWT_SECRET` (32 chars), `ALLOWED_ORIGINS` (exact frontend origins), `NODE_ENV=production`, strong `ADMIN_PASS`. The server's own boot guard will refuse to start otherwise.

2. Take and **test-restore** a database backup.
3. Walk the staging checklist in `GO_LIVE.md` 2 once on a staging/test DB (login, reload persistence, cash sale -> ledger entry, void -> stock restore, settings gating).
4. Merge `fixes/prod-hardening` -> `main` and deploy server, then client.

**Strongly recommended (not blockers):**

- Set `SENTRY_DSN` so production errors are reported the moment they happen.
- Clean the local `server/.env` (stray password line; weak dev JWT secret).
- Announce a short maintenance window - the auth system logs all active users out on deploy.

**After go-live:** first-week watch per `GO_LIVE.md` 5 (auto-close ran, backups produced, accountant reviews the first journal entries/P&L).

---

## 6. What's still missing (honest gap list)

**In-flight / partially done**

- **Multi-tenancy** - Phase 2b is partial. Read-scoping exists; full tenant isolation (write scoping everywhere, Socket.io rooms per tenant, per-tenant settings) must be completed before hosting more than one business on a single deployment. Single-business deployments are unaffected.

**Not built yet (product roadmap)**

- Multi-branch / franchise rollup reporting.
- Loyalty / customer CRM / marketing module.
- Certified hardware matrix (ESC/POS printers, cash drawers, barcode scanners) - printing works via the browser, but no formal device certification.
- Native app-store presence (currently an installable PWA).
- True server-push notifications (the "order ready" alert only fires while the customer's tab is open).

**Engineering debt (works, but raises future cost)**

- `AdminDashboard.jsx` is ~4,600 lines; tab extraction is started but the monolith remains - the main risk to fast iteration, not to correctness.
- Analytics are computed client-side (fine at current scale).
- E2E covers auth + navigation smoke; checkout/void/EOD flows are covered server-side but not yet browser-side.

**Business/process items**

- Live pilot (2-4 weeks) with real daily operations.
- Accountant sign-off on BIR/Non-VAT correctness of the generated books.
- For resale: pricing, licensing, support, and the per-instance vs multi-tenant hosting decision.

---

## 7. Bottom line

The application is feature-complete for running a single business today: selling, stock, real accounting, staff/cash control, client self-ordering, and reporting all work and are verified by 348 automated tests plus a live browser inspection with zero errors. What stands between "works" and "in production" is configuration and process - environment variables, a backup, one staging walkthrough - not code. The biggest open engineering effort is finishing multi-tenancy for the white-label/multi-client business model.